

# DIGITAL WALLET AND UPI PAYMENT SYSTEM

## *Experiment Record*

### ■ Basic Information

|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| Experiment No | 02                                                                          |
| Date          | 17/09/2026                                                                  |
| Title         | Digital Wallet and UPI Payment System using Interface and Custom Exceptions |

### ■ Objective / Aim

To design and implement a digital payment and wallet system in Java using interfaces, classes and objects, encapsulation, String validation, custom exceptions, and exception handling. The system simulates wallet creation, adding money, UPI payments, balance checking, transaction validation, and final wallet details.

### ■■ System Requirements

#### *Hardware*

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

#### *Software*

- Windows / Linux / macOS
- JDK 17 or JDK 21
- Eclipse / IntelliJ IDEA / Visual Studio Code
- Command Prompt / Terminal

### ■■ Algorithm / Logic Description

1. Create the PaymentService interface.
2. Declare pay() and checkBalance() methods.
3. Create the Wallet class with private user details and balance.
4. Create a constructor to initialize wallet details.
5. Implement addMoney() to add a valid amount.
6. Create UPIPayment implementing PaymentService.
7. Create Wallet and UPIPayment objects.
8. Validate the receiver UPI ID using String operations.
9. Throw InvalidUPIException for an invalid UPI ID.
10. Validate the payment amount.
11. Throw InvalidAmountException for zero or negative amounts.
12. Check whether sufficient wallet balance is available.
13. Throw InsufficientBalanceException when balance is insufficient.
14. Deduct the payment amount when validation succeeds.
15. Display payment success and receiver UPI ID.

16. Handle exceptions using try-catch.
17. Use finally to display transaction completion.
18. Display final wallet details and balance.

## ■ UML Class Diagram

```
PaymentService <<interface>>
-----
+ pay(upiId : String, amount : double)
+ checkBalance()
```

```
    ▲
    ■ implements
    ■
UPIPayment
```

```
-----
- wallet : Wallet
+ UPIPayment(wallet : Wallet)
+ pay(...)
+ checkBalance()
```

```
    ■ has-a
    ▼
Wallet
```

```
-----
- userName : String
- mobileNumber : String
- upiId : String
- balance : double
+ Wallet(...)
+ addMoney(amount : double)
+ getBalance() : double
+ deductMoney(amount : double)
+ displayWalletDetails()
```

```
Custom Exceptions:
InsufficientBalanceException
InvalidUPIException
InvalidAmountException
(all extend Exception)
```

Relationship: UPIPayment implements PaymentService and has a Wallet object. The three custom exceptions extend Exception.

## ■ Source Code

```
interface PaymentService {

    void pay(String upiId, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException;

    void checkBalance();
}

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

class Wallet {

    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber,
                  String upiId, double balance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    public void addMoney(double amount) {
        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Money added successfully: Rs." + amount);
        } else {
            System.out.println("Invalid amount.");
        }
    }

    public double getBalance() {
        return balance;
    }

    public void deductMoney(double amount) {
        balance = balance - amount;
    }

    public void displayWalletDetails() {
        System.out.println("\n===== WALLET DETAILS =====");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Balance      : Rs." + balance);
    }
}

class UPIPayment implements PaymentService {

    private Wallet wallet;

    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    @Override
    public void pay(String upiId, double amount)
        throws InvalidUPIException,
               InvalidAmountException,
               InsufficientBalanceException {

        if (upiId == null ||
            !upiId.contains("@") ||
            upiId.startsWith("@") ||
            upiId.endsWith("@")) {
            throw new InvalidUPIException("Invalid UPI ID.");
        }
        if (amount <= 0) {
```

```

        throw new InvalidAmountException(
            "Payment amount must be greater than zero.");
    }

    if (amount > wallet.getBalance()) {
        throw new InsufficientBalanceException(
            "Insufficient wallet balance.");
    }

    wallet.deductMoney(amount);

    System.out.println("Payment successful.");
    System.out.println("Paid Rs." + amount + " to " + upiId);
}

@Override
public void checkBalance() {
    System.out.println("Available Balance: Rs."
        + wallet.getBalance());
}
}

public class DigitalWalletDemo {

    public static void main(String[] args) {

        Wallet wallet = new Wallet(
            "Rahul",
            "9876543210",
            "rahul@upi",
            5000.0
        );

        UPIPayment payment = new UPIPayment(wallet);

        wallet.displayWalletDetails();

        System.out.println("\n===== ADD MONEY =====");
        wallet.addMoney(2000.0);

        System.out.println("\n===== CHECK BALANCE =====");
        payment.checkBalance();

        System.out.println("\n===== UPI PAYMENT =====");

        try {
            payment.pay("priya@upi", 1500.0);
        }
        catch (InvalidUPIException e) {
            System.out.println("Transaction Failed: "
                + e.getMessage());
        }
        catch (InvalidAmountException e) {
            System.out.println("Transaction Failed: "
                + e.getMessage());
        }
        catch (InsufficientBalanceException e) {
            System.out.println("Transaction Failed: "
                + e.getMessage());
        }
        finally {
            System.out.println(
                "Transaction process completed.");
        }

        System.out.println("\n===== FINAL WALLET DETAILS =====");
        wallet.displayWalletDetails();
    }
}

```

## ■ Compilation & Execution Commands

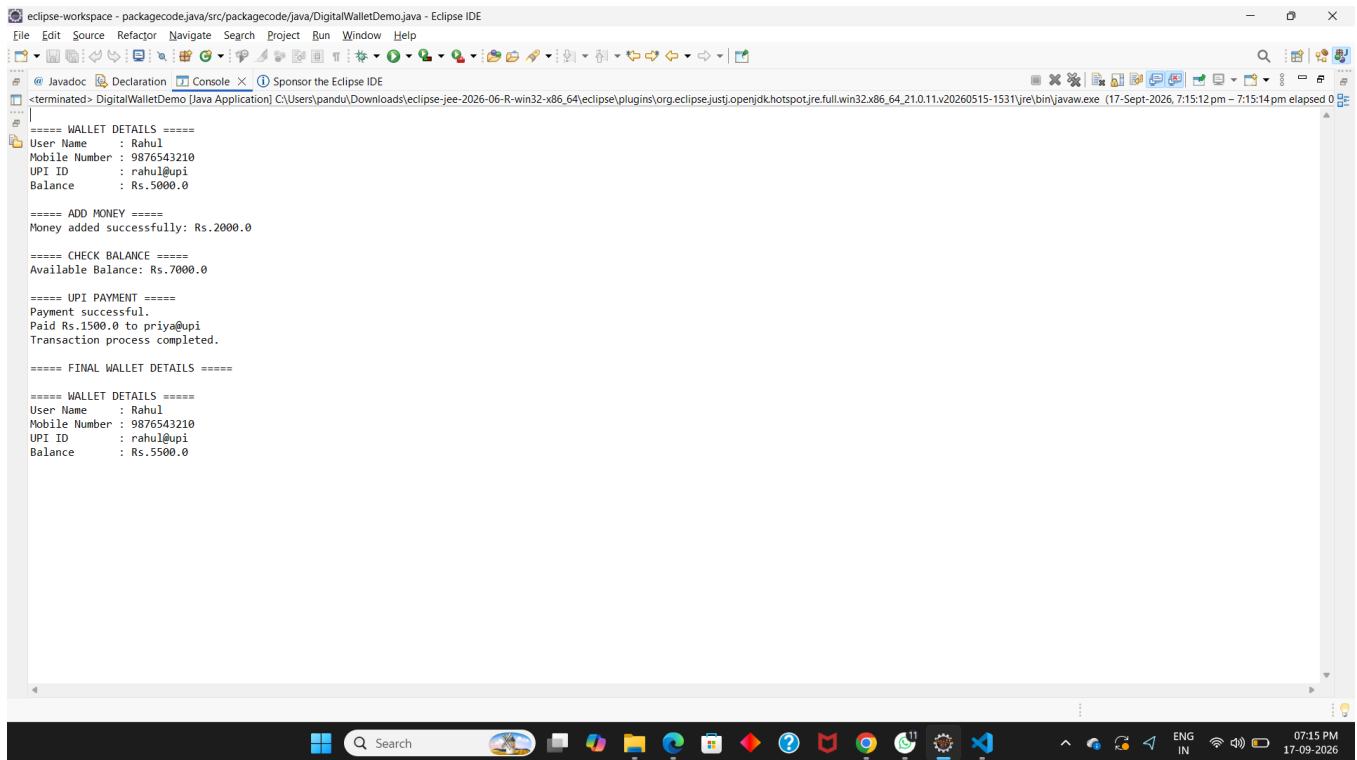
DigitalWalletDemo.java

```
javac DigitalWalletDemo.java
java DigitalWalletDemo
```

Eclipse: Right-click DigitalWalletDemo.java → Run As → Java Application

## ■ Output Screen

Eclipse Console output from the executed DigitalWalletDemo program:



```
eclipse-workspace - packagecode.java/src/packagecode.java/DigitalWalletDemo.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

<terminated> DigitalWalletDemo [Java Application] C:\Users\pandu\Downloads\eclipse-jee-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (17-Sept-2026, 7:15:12 pm - 7:15:14 pm elapsed 0

===== WALLET DETAILS =====
User Name      : Rahul
Mobile Number  : 9876543210
UPI ID         : rahul@upi
Balance        : Rs.5000.0

===== ADD MONEY =====
Money added successfully: Rs.2000.0

===== CHECK BALANCE =====
Available Balance: Rs.7000.0

===== UPI PAYMENT =====
Payment successful.
Paid Rs.1500.0 to priya@upi
Transaction process completed.

===== FINAL WALLET DETAILS =====

===== WALLET DETAILS =====
User Name      : Rahul
Mobile Number  : 9876543210
UPI ID         : rahul@upi
Balance        : Rs.5500.0
```

## ■ Observation

| S.No. | Observation        | Result                                                                    |
|-------|--------------------|---------------------------------------------------------------------------|
| 1     | Wallet Creation    | Wallet created with user name, mobile number, UPI ID and initial balance. |
| 2     | Add Money          | Specified amount added to wallet balance.                                 |
| 3     | Check Balance      | Available wallet balance displayed.                                       |
| 4     | UPI Validation     | UPI ID validated before payment.                                          |
| 5     | Amount Validation  | Invalid, zero or negative amounts identified.                             |
| 6     | Balance Validation | Payment processed only when sufficient balance is available.              |
| 7     | UPI Payment        | Payment amount deducted from wallet balance.                              |
| 8     | Exception Handling | Invalid transactions handled using custom exceptions.                     |
| 9     | Try-Catch-Finally  | Transaction exceptions handled using try-catch-finally.                   |
| 10    | Final Details      | Final wallet details and balance displayed.                               |

## ■ Result

The Digital Wallet and UPI Payment System was successfully designed and implemented using interfaces, classes and objects, encapsulation, String validation, custom exceptions, and exception handling. Wallet creation, adding money, balance checking, UPI payment, transaction validation, and final wallet details were demonstrated successfully.